TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Jorge Arun Zambrano Cedeño. Mg.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Karen Lissette Avila Moreira

PARALELO: "A"

SEMESTRE: QUINTO

ACTIVIDAD: Actividad #1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad # 1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

El estudiante deberá diseñar un sistema orientado a la gestión de casos en un refugio animal. El sistema debe permitir reportar ingresos de animales rescatados (perros, gatos, fauna silvestre), asignar voluntarios o veterinarios encargados y gestionar estados del proceso (ej. Rescatado, En Tratamiento, Adopción). Para lograr una arquitectura clara y profesional, se deberá utilizar obligatoriamente un repositorio en GitHub bajo el modelo Gitflow y aplicar al menos tres patrones de diseño de software, justificando su elección y uso técnico.

Instrucciones

Identificar los actores principales (Ciudadano/Reportante, Voluntario/Veterinario, Administrador del Refugio). Definir los requerimientos funcionales (Crear caso de rescate, categorizar especie, asignar voluntario) y no funcionales (Escalabilidad, mantenibilidad, trazabilidad de versiones).

1. Análisis del problema

El refugio animal necesita un sistema centralizado para controlar los casos de rescate desde el ingreso del animal hasta su recuperación y posible adopción.

Actores principales

Ciudadano / Reportante

- Reporta animales rescatados o en peligro.
- Proporciona ubicación y descripción.
- Consulta el estado de su reporte, si tiene autorización.

Voluntario / Veterinario

- Atiende los casos que le asignan.
- Registra observaciones y avances.
- El veterinario documenta diagnósticos y tratamientos.

Administrador del refugio

- Revisa y valida los reportes.

Asigna responsables.

Supervisa los estados de los casos.

Gestiona usuarios y genera reportes.

Requerimientos del sistema

Requerimientos funcionales

Registrar un nuevo caso de rescate.

Categorizar al animal: perro, gato o fauna silvestre.

Guardar ubicación, fecha, descripción y datos del reportante.

Asignar uno o varios responsables autorizados al caso.

Registrar diagnósticos, tratamiento y observaciones veterinarias

Consultar el historial de cambios de cada caso

Requerimientos no funcionales

Escalabilidad: permitir incorporar nuevas especies y funcionalidades.

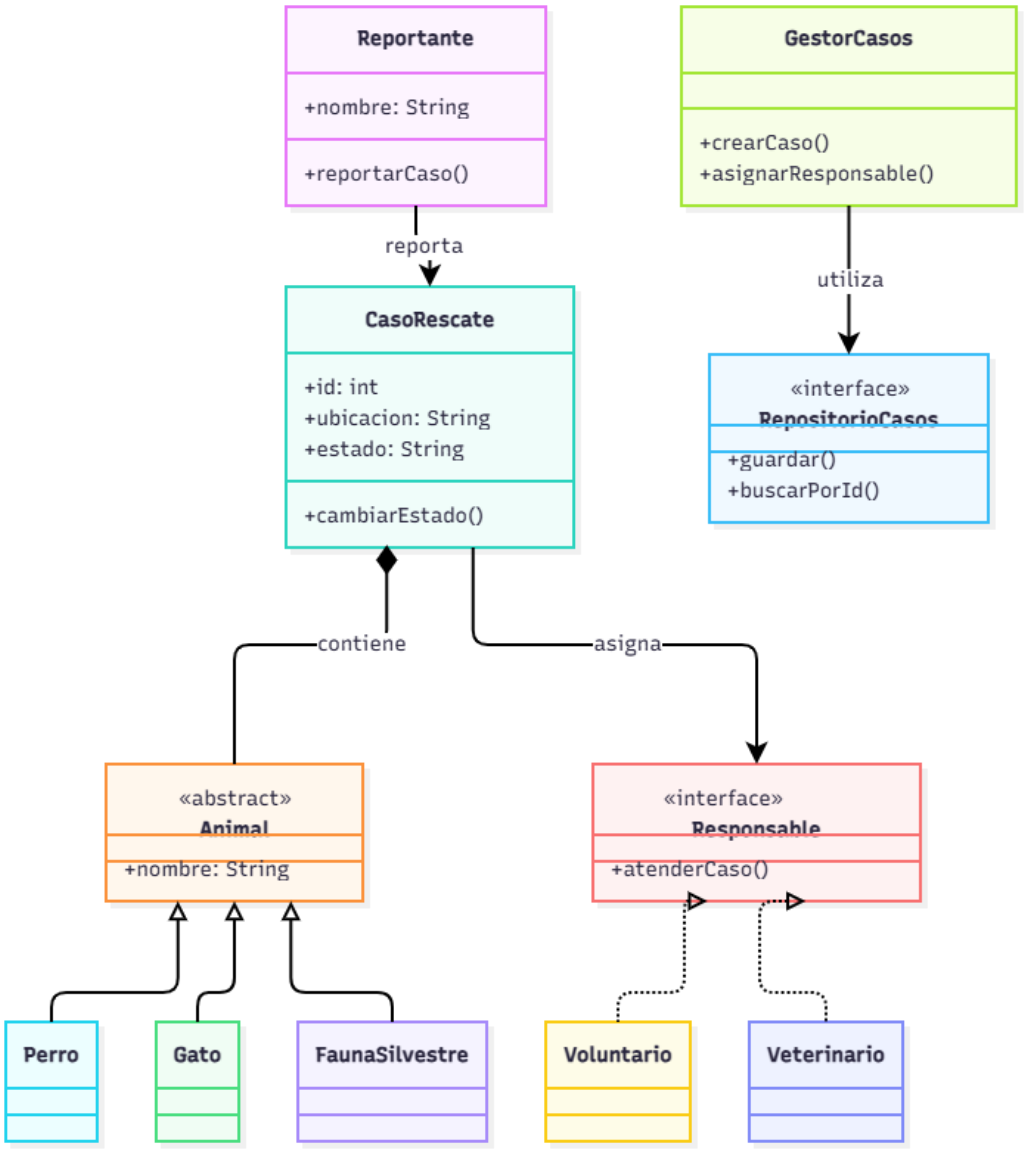
Mantenibilidad: organizar el código en clases con responsabilidades claras.

Seguridad: controlar el acceso según el rol del usuario.

Trazabilidad: registrar cambios de estado, responsables y versiones del código.

Disponibilidad e integridad: proteger los registros y evitar pérdidas o inconsistencias de información.

Diagrama de clases UML



Relaciones importantes:

Un caso de rescate tiene un expediente asociado.

El expediente es una clase abstracta con distintas implementaciones según la especie.

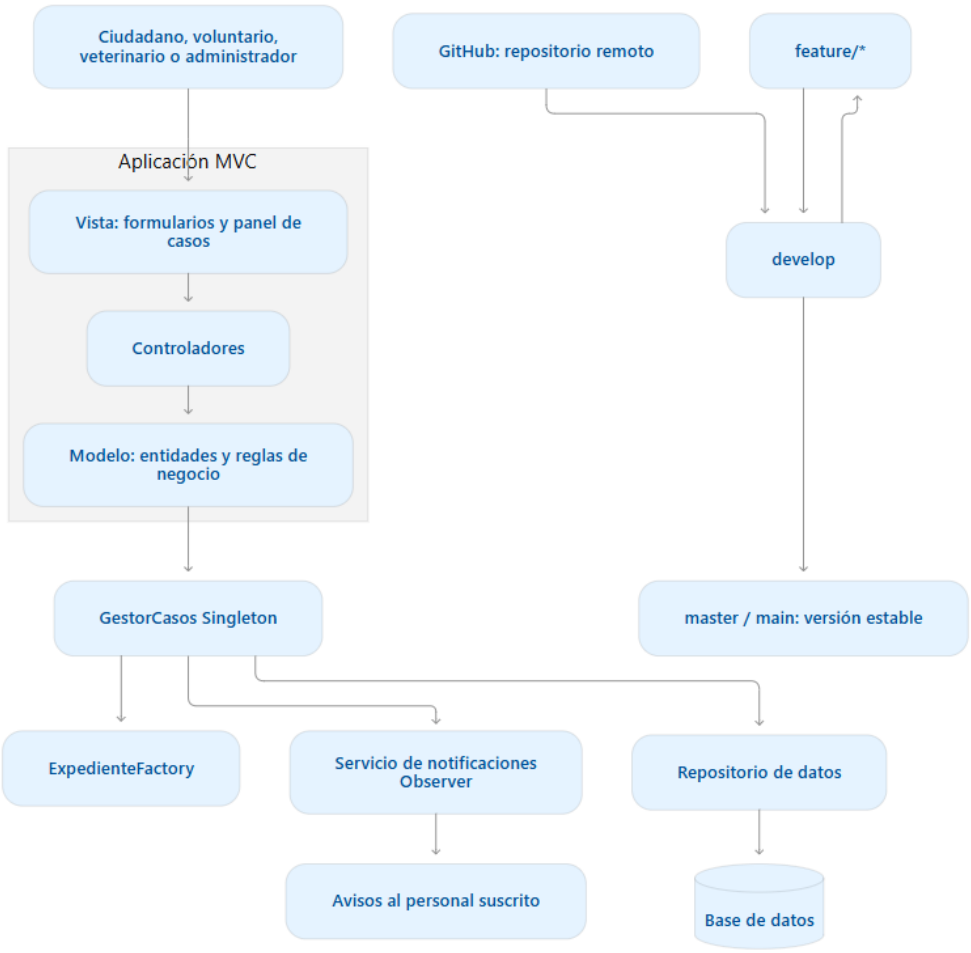
Un caso puede tener un responsable asignado; en el modelo inicial, la asignación es opcional.

El gestor central coordina el registro y la asignación de casos.

El servicio de notificaciones trabaja con observadores para avisar de nuevos rescates.

Arquitectura general del sistema

Se propone una arquitectura MVC dividida en tres capas lógicas.



El flujo funcional comienza en la vista, continúa hacia el controlador y llega al modelo, donde se aplican las reglas del negocio. El gestor central coordina las operaciones y utiliza la fábrica de expedientes y el servicio de notificaciones. La información persistente se guarda en una base de datos a través de un repositorio de acceso a datos.

GitHub no forma parte del flujo de ejecución de la aplicación: es el repositorio que controla el código fuente y sus versiones.

Patrón State

El patrón de diseño State permite modificar el comportamiento de un objeto dependiendo del estado en el que se encuentre, evitando utilizar numerosas estructuras condicionales para controlar cada situación. En el Sistema de Gestión de Casos para un Refugio Animal, este patrón se utilizará para administrar las diferentes etapas por las que pasa un animal desde que se reporta su situación hasta que se completa su proceso de atención.

Por ejemplo, un caso puede encontrarse en los estados Reportado, Rescatado, En tratamiento, En recuperación, Disponible para adopción, Adoptado o Cerrado. Cada estado tendrá reglas específicas que determinarán qué acciones se pueden realizar y cuáles serán las transiciones permitidas. De esta manera, se evitarán cambios de estado incorrectos y se garantizará un seguimiento ordenado del proceso de rescate.

Se selecciona este patrón porque el ciclo de vida de un caso de rescate requiere controlar las transiciones entre estados. Su implementación mejora la organización del código, facilita el mantenimiento de las reglas de negocio y permite incorporar nuevos estados sin concentrar toda la lógica en una única clase.

Patrón Strategy

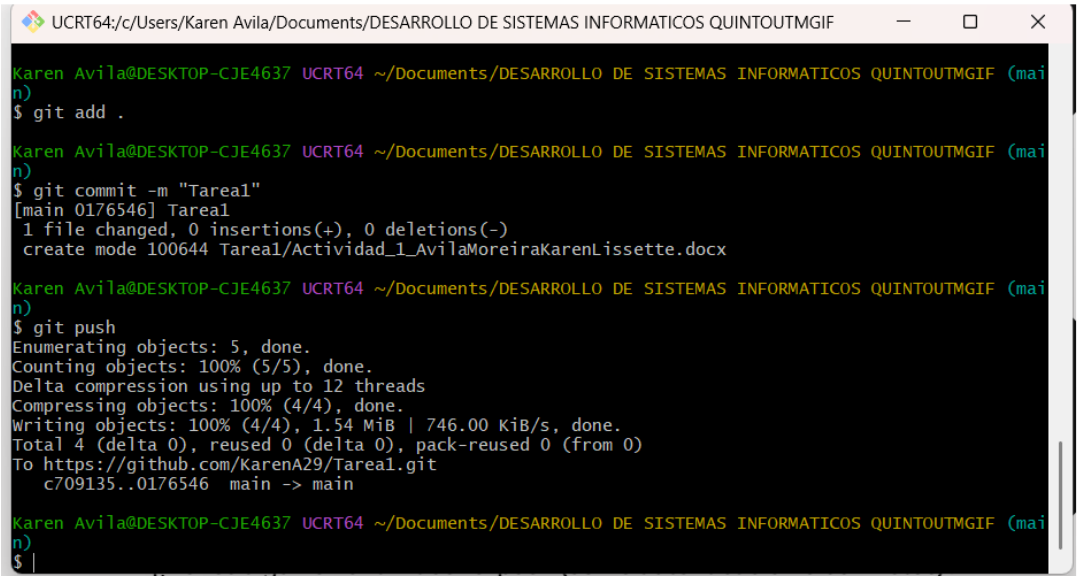
El patrón Strategy permite definir diferentes algoritmos o procedimientos para realizar una operación y seleccionar el más adecuado según las necesidades del sistema. Su principal característica es que cada estrategia se implementa de forma independiente, lo que permite cambiar el comportamiento sin modificar directamente la clase que lo utiliza.

En el Sistema de Gestión de Casos para un Refugio Animal, este patrón se puede aplicar para definir diferentes estrategias de atención según la especie y las necesidades del animal rescatado. Por ejemplo, la estrategia para perros puede contemplar una valoración veterinaria general, vacunación y desparasitación; la estrategia para gatos puede incluir evaluaciones específicas; mientras que la estrategia para fauna silvestre puede contemplar una valoración especializada y la derivación a un centro autorizado cuando corresponda.

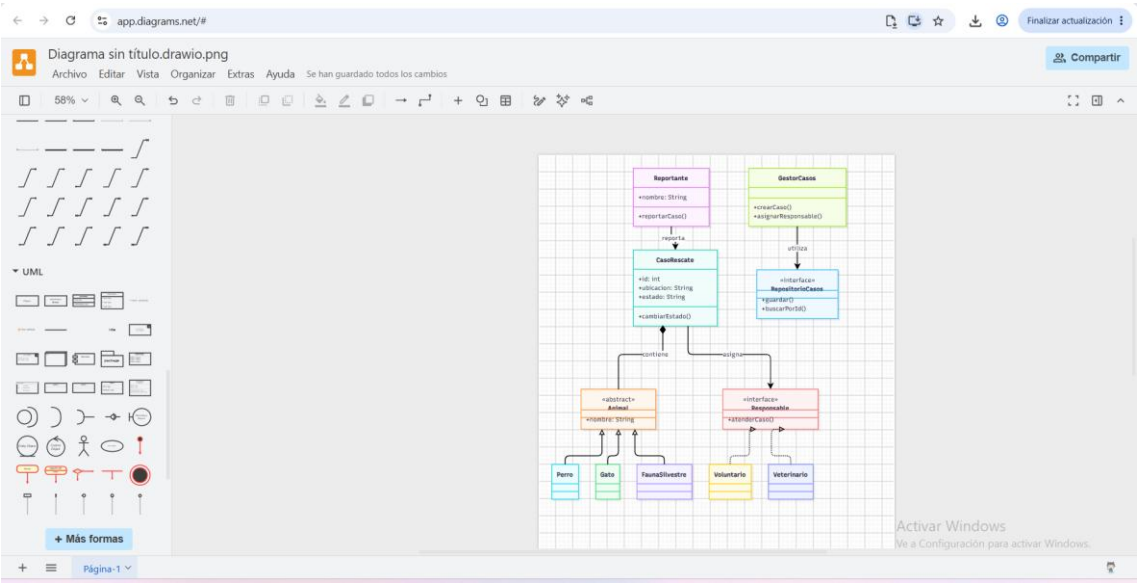
Se selecciona este patrón porque las necesidades de atención varían según las características de cada animal. Al encapsular los procedimientos en clases independientes, se reduce el uso de condicionales y se facilita la incorporación de nuevas estrategias. Además, se favorece el principio de abierto/cerrado de SOLID, ya que es posible agregar nuevos comportamientos sin modificar la estructura principal del sistema.

Evidencias

Subida del documento en el GitHub



Diseño de en Drawio



Link del repositorio de GitHub

https://github.com/KarenA29/Tarea1.git